Cloud API Design & Peer Testing — Skill Swap

Due: Apr 21 by 11:59pm

Project: Skill Swap (skill-swap490-personal)

Part 1 — Individual Cloud API (Required)

- **API Name:** CreateSwapRequest - **Purpose:** Allow a user to request a skill swap (schedule/book a lesson) with another user. - **Method:** POST - **Cloud Service:** Firebase Cloud Functions (HTTP) - **Input (JSON):** - `requesterId` (string, required) — UID of the user making the request - `providerId` (string, required) — UID of the user offering the lesson - `lessonId` (string, required) — ID of the lesson being requested - `proposedDate` (string, required) — ISO8601 datetime (with timezone) for proposed meeting - `durationMinutes` (int, optional) — length of session - `message` (string, optional) — note to provider - Auth: Bearer token (Firebase ID token) in Authorization header - **Output (JSON):** - `success` (bool) - `swapId` (string) — newly created swap document id - `status` (string) — initial status e.g., `pending` - `createdAt` (ISO8601) - `errors` (array|null) — validation details on failure - **Related Module:** `request_swap.dart` (client UI) -> Cloud Function `createSwapRequest` - **Related Database Collections/Tables:** Firestore `swaps`, `users`, `lessons`, `notifications`

Sample Request

POST /createSwapRequest Headers: - `Authorization: Bearer <Firebase ID token>` Body (JSON): json { "requesterId": "user_123", "providerId": "user_456", "lessonId": "lesson_789", "proposedDate": "2026-05-03T18:00:00Z", "durationMinutes": 60, "message": "Can we do 60 minutes on this date?" }

Sample Success Response (201)

json { "success": true, "swapId": "swap_ab12", "status": "pending", "createdAt": "2026-04-28T14:02:00Z" }

Error Cases

- 400 Bad Request — missing required field(s) - Response: 400 json { "success": false, "errors": ["Missing required field: lessonId"] }

- 401 Unauthorized — invalid or missing auth token - Response: 401 json { "success": false, "error": "Unauthorized" }

- 404 Not Found — provider or lesson does not exist - Response: 404 json { "success": false, "error": "Provider not found" }

- 409 Conflict — requested time conflicts with another confirmed booking - Response: 409 json { "success": false, "error": "Lesson not available at proposedDate" }

Part 2 — Team Cloud APIs (Minimum 2)

The team APIs below combine individual members' designs and cover core functionality for the Skill Swap app.

API 1: GetUserProfile

- **Purpose:** Retrieve public profile, skills, and rating for a given user. - **Method:** GET - **Cloud Service:** Firebase Cloud Functions (HTTP) - **Input (query parameter):** `userId` (string, required) - **Output (JSON):** `userId`, `displayName`, `avatarUrl`, `bio`, `skills` (array), `rating` (float), `memberSince` (ISO8601), `status` (`active|inactive|banned`) - **Related Module:** `user_profile.dart` - **Related DB Collections:** Firestore `users`, `ratings`

Sample Request: GET /getUserProfile?userId=user_456

Sample Response (200): json { "userId": "user_456", "displayName": "Alex Kim", "avatarUrl": "https://.../avatar.jpg", "bio": "Guitar teacher and web dev", "skills": ["Guitar","HTML","CSS"], "rating": 4.8, "memberSince": "2024-02-10T12:00:00Z", "status": "active" }

Error cases: - 400 — missing `userId` - 404 — user not found - 500 — database read error

API 2: SearchLessons

- **Purpose:** Search available lessons/swaps by keyword, skill, location, or date. - **Method:** GET - **Cloud Service:** Firebase Cloud Functions (HTTP) - **Input (query parameters):** - `q` (string, optional) — keyword - `skill` (string, optional) - `location` (string, optional) - `availableAfter` (ISO8601 string, optional) - `limit` (int, optional, default 20) - `cursor` (string, optional) — pagination token - **Output (JSON):** - `results` (array of lesson objects): `lessonId`, `providerId`, `title`, `skill`, `nextAvailable` (ISO8601), `price` - `nextCursor` (string|null) - **Related Module:** `browse.dart`, `category_results.dart` - **Related DB Collections/Indexes:** Firestore `lessons`, `lesson_search_index`

Sample Request: GET /searchLessons?skill=Guitar&availableAfter=2026-05-01T00:00:00Z&limit=10

Sample Response (200): json { "results": [{ "lessonId": "lesson_789", "providerId": "user_456", "title": "Beginner Guitar: Chords", "skill": "Guitar", "nextAvailable": "2026-05-03T18:00:00Z", "price": 0 }], "nextCursor": null }

Error cases: - 400 — invalid `availableAfter` date format - 422 — `limit` exceeds allowed maximum (e.g., >50) - 500 — search/index timeout

---

Part 3 — Request & Response (Combined Notes)

- Use consistent HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 500 Server Error. - Standardize error body shape:

json { "success": false, "errorCode": "MISSING_FIELD", "message": "lessonId is required", "details": null }

Part 4 — Peer API Testing (In-Class Activity)

Instructions for testers: - Review the assigned team's API specs. - For each API, verify clarity of input/output, logic, and error handling. - Exercise edge/error cases (missing fields, invalid formats, auth failures, high limits). - Record results using the template in Part 5.

Part 5 — Testing Results (Required)

Team Tested: Team Phoenix (example)

APIs Tested: CreateSwapRequest, SearchLessons, GetUserProfile

API: CreateSwapRequest - What is good: - Required fields are clear and response returns `swapId` and `createdAt`. - Uses standard status `pending` for new requests. - Issues found: 1. 400 responses returned HTML error page instead of JSON when missing fields (repro: send request without `lessonId`). 2. `proposedDate` accepted in multiple formats; server did not normalize timezone—caused confusion for provider scheduling. - Suggestions for improvement: - Ensure all error responses use the standardized JSON error format. - Require ISO8601 with timezone or normalize to UTC before persisting.

API: SearchLessons - What is good: - Supports pagination with `cursor` and returns `nextCursor`. - Filters (skill, date) work as expected. - Issues found: 1. When `limit=100`, server returns 422 with generic message; max limit not documented. 2. `location` matching behavior not documented (exact vs fuzzy) and returns unexpected results for partial tokens. - Suggestions: - Document `maxLimit` and return it in 422 response (e.g., `{maxLimit:50}`). - Clarify and document search matching rules.

API: GetUserProfile - What is good: - Stable schema and includes `rating` and `skills` arrays. - Issues found: 1. Requesting an inactive user returns 200 with empty `skills`; API should return `status` or 404. - Suggestions: - Add `status` field (`active|inactive|banned`) to the returned profile to avoid ambiguity.

Submission Checklist

- Individual Cloud API spec (above) - Team Cloud APIs (two provided above) - Sample Requests/Responses for each API - At least two error cases per API - Peer-testing results (example included; replace with actual class testing results) - Export to PDF or Word and submit via Canvas

---

If you'd like, I can convert this markdown to a Word `.docx` or PDF and save it as `documents/Cloud-API-Assignment.docx` or `.pdf` for direct submission. Which format do you prefer?